

CLASSIFICAÇÃO DE CARROCERIAS DE AUTOMÓVEIS

META 2 – INTELIGÊNCIA COMPUTACIONAL



Pedro Saraiva – 2023146226
Daniel Soares – 2023144661

ENQUADRAMENTO DO TRABALHO

- Aplicação de algoritmos de Inteligência Swarm à otimização de modelos.
- Estudo comparativo entre **Grey Wolf Optimization (GWO)** e **Particle Swarm Optimization (PSO)**.
- Dois objetivos principais:
 - Avaliar comportamento na função benchmark Ackley
 - Otimizar hiperparâmetros de uma CNN real para classificação de imagens

O QUE É INTELIGÊNCIA SWARM

- Paradigma inspirado em grupos naturais (lobos, aves, peixes, formigas, etc.).
- Cada agente = uma solução.
- Agentes cooperam uns com os outros e, juntos, conseguem resolver problemas de forma inteligente.
- Vantagens: método robusto, fácil de usar em paralelo, e adapta-se bem a diferentes problemas.

PSO (PARTICLE SWARM OPTIMIZATION)

- Inspirado em bandos de aves.
- Cada partícula ajusta a sua velocidade e posição com base em:
 - **pbest** — melhor posição individual
 - **gbest** — melhor posição global
- Parâmetros importantes: w , (inércia) $c1$ (coeficiente cognitivo), $c2$ (coeficiente social)
- Sensível à escolha de parâmetros.

GWO (GREY WOLF OPTIMIZER)

- Inspirado na hierarquia e caça dos lobos-cinzentos.
- Hierarquia:
 - α (Alpha) $\rightarrow$ O líder (a melhor solução encontrada)
 - β (Beta) $\rightarrow$ O 2º líder (a segunda melhor solução encontrada)
 - δ (Delta) $\rightarrow$ O 3º líder (a terceira melhor solução encontrada)
 - ω (Omega) $\rightarrow$ Restantes lobos, que seguem as ordens dos 3 líderes
- Baixa sensibilidade paramétrica.

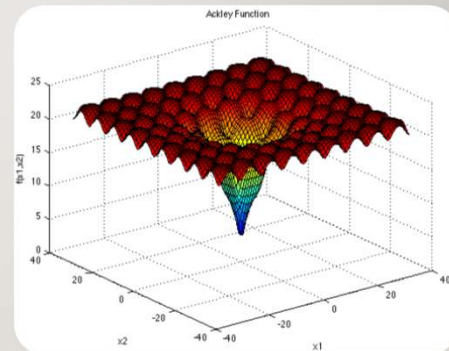
GWO - COMO FUNCIONA

O algoritmo simula uma caça em 3 passos:

1. Procurar a Presa: Os lobos espalham-se para explorar (início da otimização).
2. Encurralar a Presa: Os lobos aproximam-se da melhor área (meio da otimização).
3. Atacar a Presa: Os lobos convergem para o ponto ótimo.
 - A Posição da Presa: É estimada com base nas posições dos 3 líderes (Alpha, Beta e Delta).
 - Atualização: Cada lobo Omega atualiza a sua posição movendo-se para um ponto médio entre as posições do Alpha, Beta e Delta.

OTIMIZAÇÃO DA FUNÇÃO BENCHMARK: ACKLEY

- Função multimodal com muitos mínimos locais.
- Usada para testar estabilidade de algoritmos de otimização.
- Avaliada em $d = 2$ e $d = 3$.



RESULTADOS NO ACKLEY

- **GWO:**
 - Chegou sempre ao ótimo global (fitness = 0).
 - Desempenho estável e independente dos parâmetros.
- **PSO:**
 - Bom desempenho em alguns parâmetros.
 - Desempenho degradado com w mais elevado.
 - Sensível à escolha de $c1, c2$.

Algoritmo	Dim	Pop	Iter	w	c1	c2	Melhor Fitness	Tempo (s)
GWO	2	20	50				0	0.0176
GWO	2	50	50				0	0.0288
GWO	2	50	100				0	0.0509
PSO	2	20	50	0.40	1.50	1.50	0.000009	0.011
PSO	2	50	50	0.70	1.50	1.50	0.14306	0.0365
PSO	2	50	100	0.50	2.00	2.00	0.000011	0.0499
GWO	3	20	50				0	0.012
GWO	3	50	50				0	0.028
GWO	3	50	100				0	0.0529
PSO	3	20	50	0.40	1.50	1.50	0.000783	0.0109
PSO	3	50	50	0.70	1.50	1.50	3.731923	0.024
PSO	3	50	100	0.50	2.00	2.00	0.010683	0.0546

ARQUITETURA DA CNN INICIAL (FASE I)

- CNN simples com 3 camadas convolucionais.
- Extração limitada de padrões.
- **Accuracy no teste $\approx$ 66%.**
- Necessidade clara de otimização.

```
# arquitetura do modelo CNN
model = keras.Sequential([
    layers.Input((128,128,3)),

    layers.Conv2D(32,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(64,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(128,(3,3),activation="relu"),

    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax")
])
```

ARQUITETURA DA CNN ATUALIZADA (FASE II)

- Arquitetura aprofundada
- Mais robusta e expressiva.

```
# arquitetura do modelo CNN
model = keras.Sequential([
    layers.Input((128,128,3)),

    # Block 1
    layers.Conv2D(32, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Conv2D(32, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.3),

    # Block 2
    layers.Conv2D(64, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Conv2D(64, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    # Block 3
    layers.Conv2D(128, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    layers.Conv2D(128, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    # Classifier
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.6),
    layers.Dense(5, activation='softmax')
])
```

HIPERPARÂMETROS OTIMIZADOS

Cinco hiperparâmetros:

- **Learning Rate (0.0001 a 0.01)**
- **Kernel Size (3×3 ou 5×5)**
- **Dropout Bloco 1 (0.1 a 0.5)**
- **Dropout Bloco 2/3 (0.2 a 0.6)**
- **Dropout Camada Densa (0.3 a 0.7)**

CONFIGURAÇÃO DA OTIMIZAÇÃO

GWO:

- 10 lobos, 10 iterações.
- Treino por solução: 70 épocas, patience=7.

PSO:

- 10 partículas, 10 iterações.
- Treino por solução: 70 épocas, patience=7.
- Parâmetros: $w=0.5$, $c1=1.5$, $c2=1.5$.

RESULTADO DA OTIMIZAÇÃO: GWO

```
=====
OTIMIZAÇÃO GWO COMPLETA em 20.52 horas
Melhor Accuracy (Validação) encontrada: 0.835000

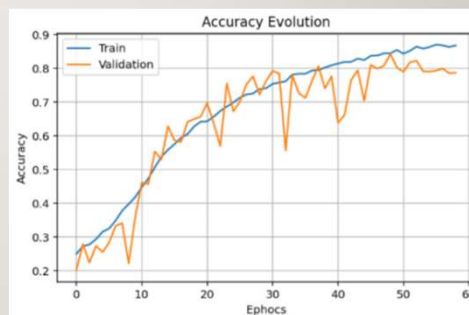
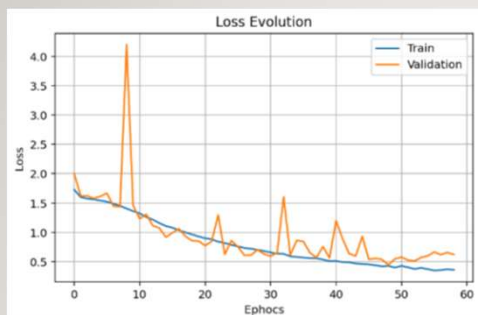
Melhor conjunto de Hiperparâmetros (lido do JSON):
f1: 32
f2: 64
f3_dense: 128
kernel: 5
lr: 0.005534
d1: 0.283087
d2_3: 0.453009
d_dense: 0.495566
=====
```

RESULTADO DA OTIMIZAÇÃO: PSO

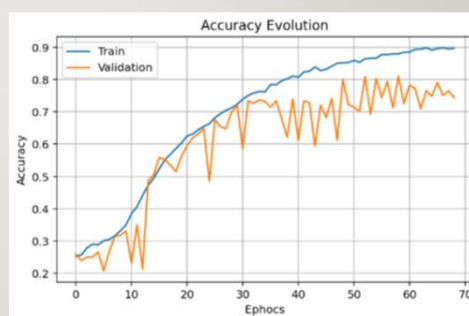
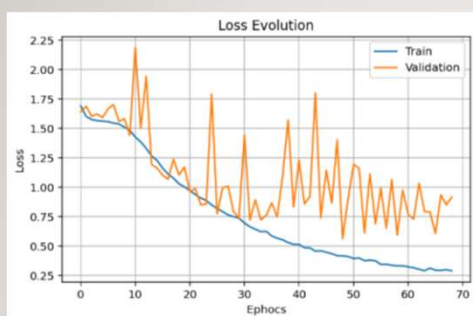
```
=====
OTIMIZAÇÃO PSO COMPLETA em 47.42 horas
Melhor Accuracy (Validação) encontrada: 0.829000

Melhor conjunto de Hiperparâmetros (lido do JSON):
f1: 32
f2: 64
f3_dense: 128
kernel: 5
lr: 0.008226
d1: 0.410817
d2_3: 0.390409
d_dense: 0.395863
=====
```

RE-TREINO COM HIPERPARÂMETROS OTIMIZADOS COM GWO



RE-TREINO COM HIPERPARÂMETROS OTIMIZADOS COM PSO



RESULTADOS FINAIS GWO

Accuracy: 0.801

Relatório (precisão, recall(sensibilidade), f-measure):

	precision	recall	f1-score	support
Convertible	0.8929	0.8750	0.8838	200
Coupe	0.6816	0.7600	0.7187	200
Hatchback	0.7979	0.7700	0.7837	200
SUV	0.9405	0.8700	0.9039	200
Sedan	0.7192	0.7300	0.7246	200
accuracy			0.8010	1000
macro avg	0.8064	0.8010	0.8029	1000
weighted avg	0.8064	0.8010	0.8029	1000

Especificidade por classe:

Convertible: 0.9738
 Coupe: 0.9113
 Hatchback: 0.9513
 SUV: 0.9862
 Sedan: 0.9287

Macro AUC (OvR): 0.9632

		Confusion matrix				
Actual	Convertible	175	24	1	0	0
	Coupe	15	152	9	0	24
	Hatchback	1	14	154	9	22
	SUV	1	2	12	174	11
	Sedan	4	31	17	2	146
		Convertible	Coupe	Hatchback	SUV	Sedan

RESULTADOS FINAIS PSO

Accuracy: 0.794

Relatório (precisão, recall(sensibilidade), f-measure):

	precision	recall	f1-score	support
Convertible	0.9471	0.8050	0.8703	200
Coupe	0.7075	0.7500	0.7282	200
Hatchback	0.8208	0.7100	0.7614	200
SUV	0.9077	0.8850	0.8962	200
Sedan	0.6560	0.8200	0.7289	200
accuracy			0.7940	1000
macro avg	0.8078	0.7940	0.7970	1000
weighted avg	0.8078	0.7940	0.7970	1000

Especificidade por classe:

Convertible: 0.9888
 Coupe: 0.9225
 Hatchback: 0.9613
 SUV: 0.9775
 Sedan: 0.8925

Macro AUC (OvR): 0.9574

	Convertible	Coupe	Hatchback	SUV	Sedan
Actual					
Convertible	161	29	4	1	5
Coupe	5	150	4	2	39
Hatchback	1	10	142	14	33
SUV	1	2	11	177	9
Sedan	2	21	12	1	164
	Convertible	Coupe	Hatchback	SUV	Sedan

COMPARAÇÃO FINAL GWO VS PSO

GWO:

- Maior estabilidade
- Menos oscilações
- Melhor convergência
- Melhor accuracy e F1-score
- Melhor capacidade de discriminação entre classes

PSO:

- Também melhorou face à CNN inicial
- Menor consistência
- Maior sensibilidade aos parâmetros

IMPACTO REAL DA OTIMIZAÇÃO

- CNN inicial: **66% accuracy**
- CNN otimizada (PSO/GWO): **79–83% accuracy**
- Melhoria muito significativa
- Algoritmos swarm encontraram hiperparâmetros impossíveis de obter manualmente
- A otimização automática mostrou ser um componente fundamental neste projeto

COMPLEXIDADE DO DATASET

- Classes de carros visualmente semelhantes → confusão natural.
- Exemplos ambíguos (ex.: descapotáveis com capota rígida fechada → parecem coupé).
- Algumas imagens mal catalogadas (SUV na pasta Coupé).
- Ruído: texto sobreposto, ângulos estranhos, objetos a tapar o carro.
- Dificulta alcançar +90% accuracy.
- Até à fase 3 → limpeza e filtragem do dataset (diminuição do nº de imagens).

CONCLUSÕES GERAIS

- GWO apresentou desempenho superior e mais robusto.
- PSO melhorou, mas foi menos consistente.
- Ambos os métodos melhoraram drasticamente a CNN inicial.
- Swarm Intelligence mostrou grande eficácia para otimização de hiperparâmetros.